EVFLOW — Kishore's Backend Scope

EV trip planner, Bengaluru → Mysuru demo corridor · 5-day hackathon · Hari = frontend (Next.js web), Kishore = backend (Spring Boot + PostgreSQL) · 27 Jul 2026

The deal: we only touch each other at the 8 REST endpoints below. Hari builds against mocked versions of these until yours are live. The contract in this doc is **frozen**: any change to a URL or JSON shape has to be agreed by both of us, otherwise integration day breaks.

1. What you own (4 modules, down from 6)

Module	What it does	Effort
Vehicle	Seeded EV profiles + optional custom profile create	Easy CRUD
Charger	Charger DB seeded from Open Charge Map, bounding-box query	CRUD + seed script
Route	`POST /api/route/plan`: energy model + charging-stop insertion. **The core IP. The one hard endpoint.**	The real work
Trip	Start trip, complete trip, history + summary	Easy CRUD

Cut from your original diagram (on purpose):

- **Maps Module:** frontend calls Google Maps / Directions / Elevation / Places directly with a restricted browser key. Your backend never touches Google APIs. Frontend sends you the route points it already has.
- **AI Assistant Module:** the assistant is an LLM call living in the frontend layer whose "tools" are your existing endpoints. Zero new backend work.
- **PostGIS:** optional. Demo scale is one corridor with ~20 curated chargers; plain lat/lng columns + bounding-box `WHERE` clause is enough. Add PostGIS later if you want it, nothing in the contract changes.

2. Database schema

```
vehicles  (id, name, battery_kwh, base_wh_per_km, connector_type, max_charge_kw)
chargers  (id, name, lat, lng, power_kw, connector_types text[], operator, address, verified
bool)
trips     (id, vehicle_id, origin_name, dest_name, distance_km, energy_kwh,
           cost_inr, started_at, ended_at)
```

Seed vehicles (approx real-world figures from spec sheets; tune freely):

Name	battery_kwh	base_wh_per_km	connector	max_charge_kw
Tata Nexon EV LR	40.5	145	CCS2	50
MG ZS EV	50.3	170	CCS2	76
Tata Tiago EV	24.0	110	CCS2	25

3. API contract (frozen)

All JSON. Base path `/api`. Errors, every non-2xx response:

```
{ "error": { "code": "ROUTE_INFEASIBLE", "message": "human readable" } }
```

#	Endpoint	Purpose
1	`GET` `/api/vehicles`	List vehicle profiles
2	`POST` `/api/vehicles`	Create custom profile (same fields as schema)
3	`GET` `/api/chargers?minLat=&minLng=&maxLat=&maxLng=&connector=CCS2`	Chargers in bounding box, optional connector filter
4	`GET` `/api/chargers/{id}`	Charger detail
5	`POST` `/api/route/plan`	**Energy model + stop insertion (below)**
6	`POST` `/api/trips`	Start trip → `{ "id": 7, "startedAt": "…" }`
7	`POST` `/api/trips/{id}/complete`	End trip → summary (distanceKm, energyKwh, costInr, durationMin, avgWhPerKm)
8	`GET` `/api/trips`	Trip history, newest first

Charger JSON shape (used everywhere a charger appears)

```
{ "id": 12, "name": "Tata Power EZ Charge – Maddur", "lat": 12.5843, "lng": 77.0461,
  "powerKw": 50, "connectorTypes": ["CCS2"], "operator": "Tata Power",
  "address": "NH275, Maddur", "verified": true }
```

4. The hard endpoint: POST /api/route/plan

Frontend gets the route polyline from Google Directions and elevation from Google Elevation, then sends you sampled points (~1 per 2 km, ≤150 points):

Request

```
{
  "vehicleId": 1,
  "startBatteryPercent": 90,
  "points": [
    { "lat": 12.9716, "lng": 77.5946, "distanceKm": 0.0,  "elevationM": 920 },
    { "lat": 12.9502, "lng": 77.5720, "distanceKm": 2.1,  "elevationM": 905 },
    { "lat": 12.3052, "lng": 76.6551, "distanceKm": 143.2, "elevationM": 763 }
  ]
}
```

Energy model (simple and calibratable; do NOT overbuild)

```
// per segment between consecutive points:
gainM           = max(0, elev[i+1] − elev[i])          // ignore descents (conservative)
segmentWh       = segKm * vehicle.baseWhPerKm  +  gainM * CLIMB_WH_PER_M
CLIMB_WH_PER_M  = 6.0    // calibration constant, tune against a real trip later

batteryPercent −= segmentWh / (vehicle.batteryKwh * 1000) * 100
```

Charging-stop insertion rule (make it match the pitch: "30% rule")

```
walk the curve point by point:
  if batteryPercent < 30 at this point
     and no already-planned stop within the NEXT 20 km:
       find compatible chargers (connector match) within 5 km of any route
       point in the last 30 km; pick highest powerKw
       if none → return feasible=false + error code ROUTE_INFEASIBLE
       insert stop there, charge to 80%, recompute curve from that point on

estChargeMinutes = (targetKwh − arrivalKwh) / min(charger.powerKw, vehicle.maxChargeKw)
                   / 0.9 * 60        // 0.9 = charging efficiency
```

Response

```
{
  "feasible": true,
  "totalDistanceKm": 143.2,
  "totalEnergyKwh": 21.4,
  "batteryCurve": [
    { "distanceKm": 0.0,  "batteryPercent": 90.0 },
    { "distanceKm": 2.1,  "batteryPercent": 89.2 }
  ],
  "chargingStops": [
    { "charger": { …charger JSON shape… },
      "atDistanceKm": 96.5, "arrivalBatteryPercent": 28.4,
      "chargeToPercent": 80, "estChargeMinutes": 34 }
```

```
  ]
}
```

Trip cost on `/complete`: `costInr = energyKwh × 18` (₹18/kWh flat; constant at top of file, good enough for demo).

5. Charger data seeding (Day 1 afternoon, highest-value non-code task)

- Register free API key at **openchargemap.org** (instant, no approval wait).
- One-off script (Java or even curl + SQL) pulling the Bengaluru–Mysuru corridor: `https://api.openchargemap.io/v3/poi?output=json&countrycode=IN&boundingbox=(12.2,76.5),(13.1,77.8)&maxresults=500&key=YOUR_KEY`
- Normalize into `chargers` table (map their connection types → `CCS2 / CHAdeMO / Type2 / AC`).
- **Manually verify and fix 15–20 chargers actually on NH275** (Google Maps cross-check), set `verified=true`. OCM data in India is stale in places, and one wrong charger pin in the demo kills credibility. This hour matters more than any code that day.

6. Hosting + CORS (Day 1 morning, before anything else)

- Deploy skeleton Spring Boot app + Postgres to **Railway** (or Render) free tier Day 1. Prove the deploy pipeline works Day 1, not Day 4. Send Hari the base URL immediately.
- CORS. Without this the frontend cannot call you at all:

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {
  @Override public void addCorsMappings(CorsRegistry r) {
    r.addMapping("/api/**")
     .allowedOrigins("http://localhost:3000", "https://YOUR-VERCEL-APP.vercel.app")
     .allowedMethods("GET", "POST", "PATCH", "DELETE");
  }
}
```

> **Keep the free-tier instance awake** during the demo (Render free tier sleeps after idle; hit it 5 min before presenting, or use Railway).

7. Your day-by-day plan

Day	Deliverable (deployed, not just local)
1	Spring Boot skeleton + Postgres live on Railway. CORS on. `GET /api/vehicles` and `GET /api/chargers` returning seeded data. OCM seed done + 15-20 corridor chargers hand-verified. **Send Hari the base URL.**
2	`POST /api/route/plan` working: energy model + 30%-rule stop insertion. One unit test with a fabricated 150 km route proving a stop gets inserted. Hari integrates the same evening.
3	Trip endpoints (start / complete / history). Error shapes everywhere. Stable deploy.
4	**No new features.** Bug-fix whatever integration shakes out. Server stays up and boring.

5	Buffer + pitch. Run the demo flow 5+ times back to back.

8. Definition of done

Hari opens the live Vercel app on a phone → picks Nexon EV → plans Bengaluru → Mysuru → your `/route/plan` returns a battery curve that dips below 30% around Maddur and inserts a real, hand-verified charger → completes the trip → your trip summary comes back with cost in ₹. Everything else is frontend.